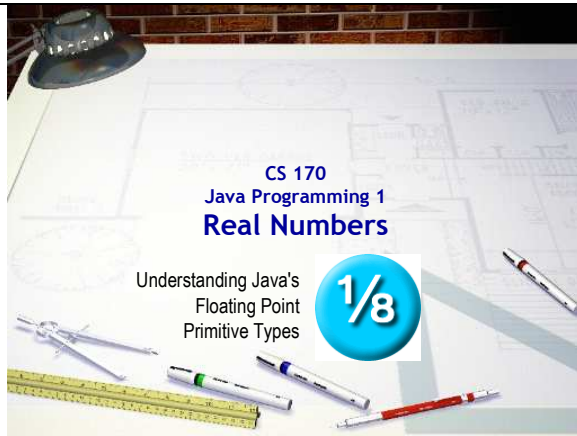




Slide 1

CS 170

Java Programming 1

Real Numbers

Duration: 00:00:54

Advance mode: Auto

Notes:

Hello there. This is the CS 170, Java Programming 1 lecture, Real Numbers

As you saw in class this week, Java has both object types, such as Graphics and String, and primitive types, which consist of numbers, characters, and true-false values. During the lecture, you met the four Java integer types: byte, short, int, and long. Each integer type represents a set of signed, whole numbers of a particular range; byte has the smallest range, and long has the largest. None of these integer types, however, can represent fractional or "real" numbers.

Real numbers in Java are represented by the two floating-point types, float and double. Floating-point primitive types can hold fractional values in addition to whole numbers. These types are the topic of this lesson.

- Floating-point types
 - Can hold a **fractional** amount
- Binary value is divided into three parts
 - A binary fraction, called the **mantissa**
 - A signed binary integer called the **exponent**
 - A single bit used to represent the **sign**

Slide 2

Real (floating-point) Numbers

Duration: 00:00:59

Advance mode: Auto

Notes:

As you learned in lecture, even though you normally use decimal or base-10 numbers when you write your Java programs, internally those numbers are stored in binary. Java also has floating-point numbers which can hold a fractional amount like one-eighth or 2 and $\frac{1}{2}$.

Floating-point numbers can store fractional values because they use a different internal format than binary integers. Instead of storing one simple binary value, floating-point numbers are stored as two binary numbers: a binary fraction known as the significand or, more informally, the mantissa along with signed integer value, called the exponent. In addition, floating-point numbers keep one bit reserved to store the sign of the number.

When the actual value is used, a mathematical operation is performed by your computer using the two binary numbers to compute the final amount.

- How can we convert 13.75 to a floating-point number?
 - Step 1: convert the integer part to a binary number
1101 - $1 \times 8 + 1 \times 4 + 1 \times 1$
 - Step 2: convert the fractional part to a binary number
.11 : $1 \times \frac{1}{2} + 1 \times \frac{1}{4}$
 - Step 3: calculate the exponent
 - Need to move the "binary" point left by 4
 - Mantissa is **110111** and exponent is **100**
- Simplified; actual storage is slightly more complex
 - http://en.wikipedia.org/wiki/Floating_point

Notes:

Here's an example. Suppose we want to store the real number 13.75 as a floating point number. How would you do that?

First, you'd need to convert the whole number part to the binary number 1101 the same way you learned in class this week: that's one eight, one four, no twos and one one.

But, how do we represent the .75 part? The fractional portion of a binary number works just the same way as the fractional portion of a decimal number. In a decimal number the first place to the right of the decimal is 1/10, the next 1/100, and the next 1/1000. With the binary, or base 2 system, the first

Slide 3

A Floating Point Example

Duration: 00:01:53

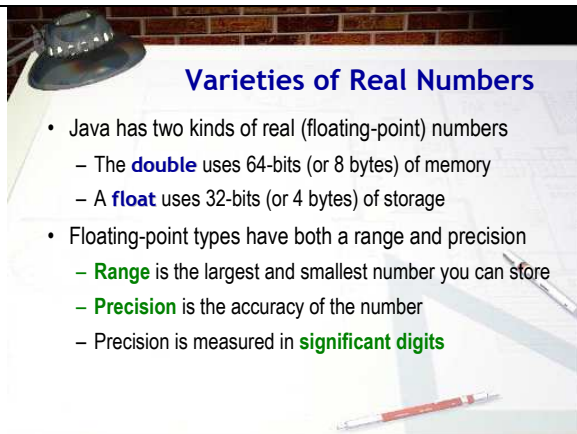
Advance mode: Auto

position to the right is $1/2$, the next $1/4$, and the following $1/8$.

Given this information, you can see that we can represent the decimal number 13.75 as the binary number 1101.11, where the .11 represents $1/2 + 1/4$ or .75 in decimal.

To store this number in the floating-point format, we move the binary point to the left by 4 places. (It's not a decimal point because it's not a decimal number. Technically, it's the radix point.) In the end, we have a mantissa of 110111 and an exponent of 100 (4 in decimal).

This is actually kind of a simplified example. If you're interested, you can learn more about how floating-point numbers are stored by reading the Wikipedia article on floating-point numbers.



Varieties of Real Numbers

- Java has two kinds of real (floating-point) numbers
 - The **double** uses 64-bits (or 8 bytes) of memory
 - A **float** uses 32-bits (or 4 bytes) of storage
- Floating-point types have both a range and precision
 - **Range** is the largest and smallest number you can store
 - **Precision** is the accuracy of the number
 - Precision is measured in **significant digits**

Slide 4

Varieties of Real Numbers

Duration: 00:01:23

Advance mode: Auto

Notes:

As you saw in lecture, Java has four different kinds of integers. However, it has only two different primitive types for storing floating-point numbers, the float and the double. As with the different types in the integer family, each of these two types uses a different amount of memory and can represent a different range of numbers.

The double, (which is the default format for floating-point literals, just like int is the default format for integer literals), uses eight bytes (64 bits) of storage. The float uses half the storage as a double, at four bytes (32 bits).

While the different integer types differ only in their range, floating-point types differ in their precision or accuracy as well. In other words, even though a byte can only represent 256 different numbers, each number is 100% accurate. You don't gain any more precision by storing a number in a long variable than you do in a byte; you just get a larger range of numbers.

Floating-point numbers are potentially less accurate, depending on the number you're trying to represent. You measure the precision of a floating point number by referring to the number of significant digits in its representation.

Let's see exactly what that means.

[illegible]

- ## Slide 5

Range and Precision

Duration: 00:02:16

Advance mode: Auto

Notes:

Let's start by looking at the float type. The float and int type, you remember, both use 32 bits. Using 32 bits, the int type can represent values between (roughly) 2 billion and -2 billion or $2.15E+9$. With those same 32 bits, though, the float can represent a range of numbers between $\pm 3.4E+38$, that is, 34 with 37 decimals behind it. That's a much larger range than an int can store. (I had to look it up, but that's plus or minus 340 undecillion). While the int can store the values 0 and 1, the float can store a whole range of values that are less than 1 but greater than zero. Specifically, it can store numbers as small as $1.4E-45$, which is a really small number, as you can see here.

So, what's the catch? Accuracy or precision. Although you can store a really big number using a float, only the first eight digits have any meaning at all. All of the digits past the first eight are just set to 0, no matter what you type in your source code. When you learn how to write floating-point literals, we'll use Code Pad to look at some examples.

The range of numbers the double type can represent include the positive and negative numbers as small as 4.94E-324, and positive or negative numbers as large as 1.79E+308. The precision for a double is 16 or 17 significant digits, depending on the number.

As you can see, the double type has a range approximately ten times that of the float, although its precision is only about twice the float. That is the meaning of the name double, which is short for double precision. For the same reason, some other languages, such as Visual Basic, call the float type, a single.

Defining Real Variables

- Use **double** or **float** for type name
- Floating-point literals can be written in two ways
 - Decimal notation: **3.45**, **.7653**, **2.0**

double first = .234;
double second = 7.5432;
float third = 3.2F;



- Must** use a suffix of **F** or **f** when writing a float literal
- May** use a suffix of **D** or **d** when writing a double literal

- ## Slide 6

Defining Real Variables

Duration: 00:01:55

Advance mode: Auto

Notes:

Just as with integers, you create floating-point variables by specifying the type, (float or double), and a name, optionally followed by an initializer. As with the integers, you can use to write floating-point literals, previously initialized variables, expressions, or user-input methods to initialize your floating-point variables.

You can write floating-point literals in two different ways:

- using decimal notation, or
- using exponential, (or scientific), notation.

When you use decimal notation, you may use an optional sign, followed by digits and a single decimal point. You may have digits on either side (or both sides) of the decimal-point, but you don't have to have digits on both sides of the decimal, as you do in some languages (such as Pascal).

These are both valid variables, initialized using floating-point literals and written using decimal notation.

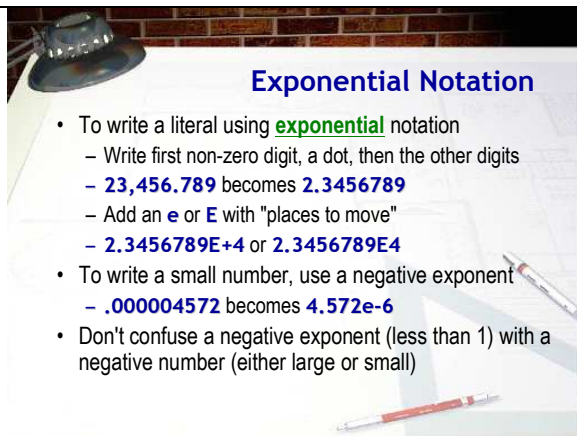
When you write a floating-point literal, Java automatically uses the double type. This creates a problem when you want to

store the number in a float variable.

Remember that floating-point numbers are stored in a different format than binary integers. To store a double value in a float variable, Java cannot simply copy part of the number, as it does for integers. Instead, Java has to re-compute both the mantissa and the exponent portion of the number, which it does not do automatically.

Because of this, when you want to store a literal floating-point number in a float variable, you must add an F at the end of the number like this.

Note that you may use either an uppercase F or a lowercase f. You may also append an uppercase D or a lowercase d to a literal to signify that it is of type double. This is occasionally useful: the literal 7, for instance, is an int, while 7d and 7.0 are both double.



Exponential Notation

- To write a literal using **exponential** notation
 - Write first non-zero digit, a dot, then the other digits
 - **23,456.789** becomes **2.3456789**
 - Add an **e** or **E** with "places to move"
 - **2.3456789E+4** or **2.3456789E4**- To write a small number, use a negative exponent
 - **.000004572** becomes **4.572e-6**
- Don't confuse a negative exponent (less than 1) with a negative number (either large or small)

Slide 7

Exponential Notation

Duration: 00:01:60

Advance mode: Auto

Notes:

You can also write floating-point literals using exponential, (aka scientific), notation. To use exponential notation, write the first digit of your number, followed by a decimal point and all of the rest of your digits.

For instance, to write the number 23456.789 in exponential notation, start by writing:

2.3456789

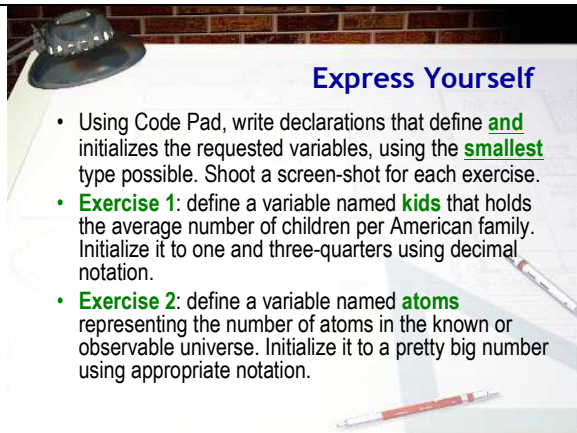
Immediately following this, write an e or an E—making sure you don't add any spaces—followed by the number of places required to move the decimal when the number is "decoded."

In our example, we need to move the decimal to the right by four places to reconstruct our original, so the finished number would look like this:

2.3456789E4

If the number you're working with is small—say .000004572—you do the same thing. The only difference is you use a negative exponent which says, "move the decimal point to the left, not the right". To write the preceding number in exponential notation you'd write: 4.572e-6

Make sure you don't confuse using a negative exponent (where the sign goes after the E/e) and a negative number, where the sign goes before the first digit. If a negative number has a positive exponent, the number is less than -1. If a negative number has a negative exponent, it is between 0 and -1. If a positive number has a negative exponent it is between 0 and +1, and if a positive number has a positive exponent it is greater than +1.



Express Yourself

- Using Code Pad, write declarations that define **and** initializes the requested variables, using the **smallest** type possible. Shoot a screen-shot for each exercise.
- Exercise 1:** define a variable named **kids** that holds the average number of children per American family. Initialize it to one and three-quarters using decimal notation.
- Exercise 2:** define a variable named **atoms** representing the number of atoms in the known or observable universe. Initialize it to a pretty big number using appropriate notation.

Slide 8

Express Yourself

Duration: 00:01:16

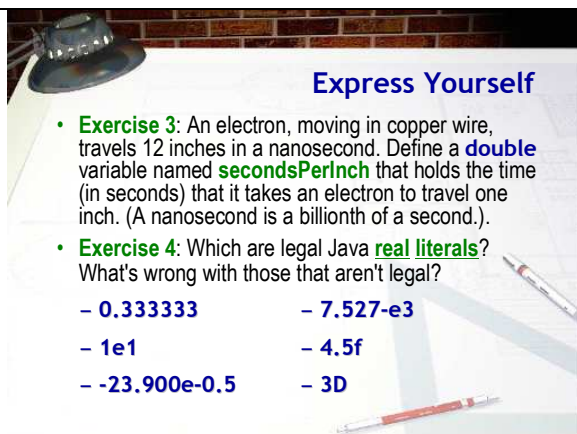
Advance mode: Auto

Notes:

OK, now it's your turn. Start a new section in your lab exercises document this week and then open up BlueJ. Using Code Pad, write the declarations I've requested below using the smallest type possible. Shoot a screen-shot for each exercise.

For Exercise 1, define a variable named kids that holds the average number of children per American family. Initialize it to one and three-quarters using decimal notation. What I want to see here is that you know how to select the correct type and how to write literals of that type. After you've defined your variable in Code Pad, simply type its name on the next line to display it's value and type.

For Exercise 2, define a variable named atoms representing the number of atoms in the known or observable universe. You can look up the number (which is pretty big and pretty approximate). Use an appropriate notation. When you're done, type the name of your variable into Code Pad, just like you did for Exercise 1, so I can see the type and value.



Express Yourself

- Exercise 3:** An electron, moving in copper wire, travels 12 inches in a nanosecond. Define a **double** variable named **secondsPerInch** that holds the time (in seconds) that it takes an electron to travel one inch. (A nanosecond is a billionth of a second.).
- Exercise 4:** Which are legal Java **real literals**? What's wrong with those that aren't legal?

- 0.333333	- 7.527-e3
- 1e1	- 4.5f
- -23.900e-0.5	- 3D

Slide 9

Express Yourself

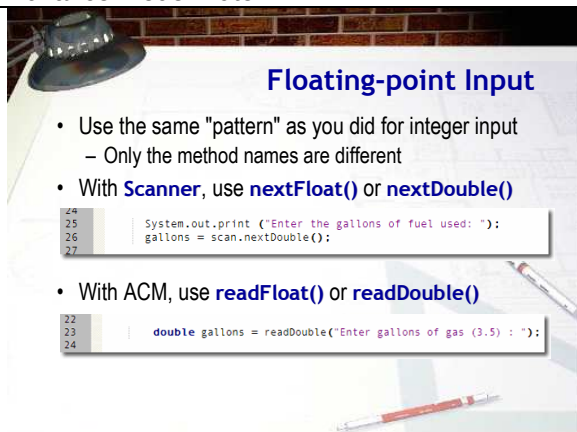
Duration: 00:00:50

Advance mode: Auto

Notes:

For Exercise 3, I want you to define a double variable, named secondsPerInch that contains the time, in seconds, that it takes an electron to travel one inch along a copper wire. An electron will travel 12 inches in a nanosecond, and a nanosecond is one billionth of a second. Print the value of the variable when you're done, just as with the previous examples.

For Exercise 4, look at these symbols. Which of them are real-number literals and which are not. For those that are not real-number literals, tell me why not. Simply type your answers into your exercise document.



Floating-point Input

- Use the same "pattern" as you did for integer input
 - Only the method names are different
- With **Scanner**, use **nextFloat()** or **nextDouble()**

```

24
25 System.out.print ("Enter the gallons of fuel used: ");
26 gallons = scan.nextDouble();
27

```

- With ACM, use **readFloat()** or **readDouble()**

```

22
23 double gallons = readDouble("Enter gallons of gas (3.5) : ");
24

```

Slide 10

Notes:

OK, we're almost ready to start writing programs using real numbers. To do that, we have to cover a couple of details.

The first thing we'll want to look at is input. You know how to get String and integer input from the user (using both the Scanner class and the ACM Console programs), but how do you get floating point input.

It turns out that the pattern or recipe for floating point input is almost exactly the same as what you used for String and integer input; only the method names are different.

For traditional, plain console programs, where you use the Scanner class, instead of using **nextLine()** or **nextInt()**, use

Floating-point Input

Duration: 00:01:11

Advance mode: Auto

nextFloat() or nextDouble() instead, just like this example.

If you are writing an ACMConsole program, things are even easier. You use the readFloat() or readDouble() methods. Just like the ACM readInt() method, you supply the prompt you want to appear as a parameter when you call readDouble().



Express Yourself

- **Exercise 5:** Create a BlueJ project to hold this week's exercises and then create a new ACM Console Program. Create a double variable named price. Ask the user to enter the price of a hamburger, and store the result in your variable. Now, create a second int variable called pennies. Multiply the price by 100, and store in pennies. Print the number of pennies.
- What happens when you compile? Print a screen-shot of your code along with BlueJ's error message.

Slide 11

Express Yourself

Duration: 00:01:28

Advance mode: Auto

Notes:

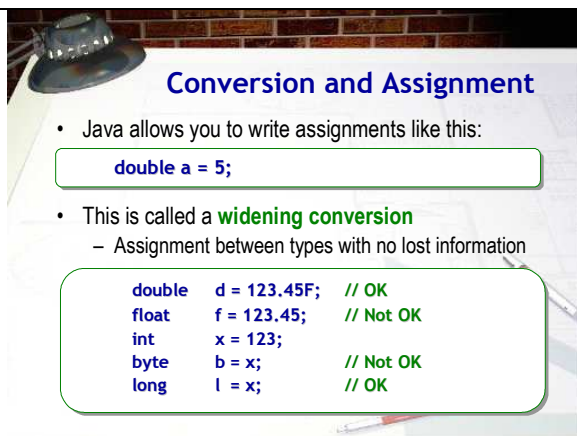
OK, now let's write an interactive program that uses floating-point numbers, similar to the program we ended the in-class lecture with this week. Start a new BlueJ project for this week's exercises (I usually try to have my projects match the name of my in-class lab exercise document), and create a new ACM Console program.

For this program, we're going to ask the user to enter the price of a hamburger and then print out how many pennies he has to give us to pay for it. (In other words, we're going to convert a floating point value like \$4.35 to an integer value like 435 pennies.) Here are the steps to follow.

- First, create a double variable named price.
- Then ask the user to enter the price of a hamburger and store the user's response in your variable.
- Next, create an int variable named pennies.
- Multiply the price times 100 and store the result in pennies.
- Print out the number of pennies along with a message.

When you try to compile, you should get an error message. Print a screenshot of your code along with the error message displayed at the bottom of the editor window.

Now, let's see why we get that error.



Conversion and Assignment

- Java allows you to write assignments like this:

```
double a = 5;
```
- This is called a **widening conversion**
 - Assignment between types with no lost information

double	d = 123.45F;	// OK
float	f = 123.45;	// Not OK
int	x = 123;	
byte	b = x;	// Not OK
long	l = x;	// OK

Slide 12

Conversion and Assignment

Notes:

The problem in your program is simple: price is a double and pennies, the variable where you are trying to store the result, is an int. Suppose the situation was reversed, though, suppose that price were an int and pennies as a double. In that case, you'd have no problem. Java allows you to write assignments like this.

This is called a widening conversion because the variable where you are storing the data is "wider" than the size of the data itself. When you try to store the integer value 5 in the double variable a, Java is able to automatically create a suitable double value, 5.0 to place inside the variable. In a widening conversion like this, no information is lost.

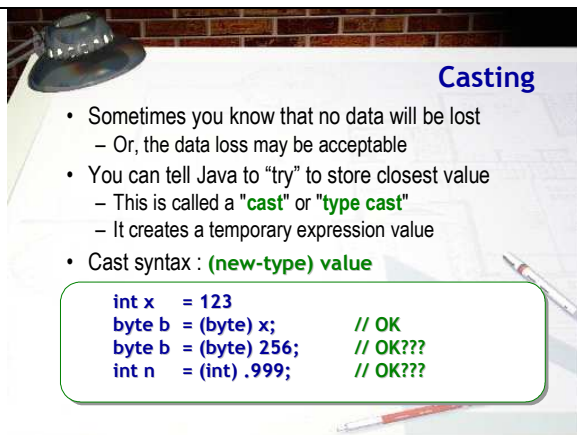
Duration: 00:02:21
Advance mode: Auto

Here are some other assignments.

- Note when we try to store a float value in a double variable that's perfectly OK. When the float value is converted to a double, it will be at least as accurate as when it was stored in a float.
- The reverse, though, is NOT OK. If you try to store a double value into a float variable, Java has no way of telling if the range of your value will fit inside a float. Remember the range for doubles is almost ten times larger than that for floats. Even if the range was OK, though, the precision of the double value is twice that of a float, so you could still lose information. For that reason, Java prohibits it.

Here's another example. Suppose I have an int variable x and I store 123 in it. You might think that I could then assign x to a byte variable b, since 123 is certainly in the range of valid bytes. That's not what happens though. Java doesn't look at the value I have stored in the variable x, is simply looks at the type and notes that the value stored in x could easily exceed the range of a byte. It won't try to store the wider (potential) value into the narrower actual container, even though the actual value in this case would fit.

If I go the other way, though, and try to store x's value in a wider container, like the long l, that's just fine.



Casting

- Sometimes you know that no data will be lost
 - Or, the data loss may be acceptable
- You can tell Java to "try" to store closest value
 - This is called a "cast" or "type cast"
 - It creates a temporary expression value
- Cast syntax : **(new-type) value**

```
int x = 123
byte b = (byte) x;      // OK
byte b = (byte) 256;    // OK???
int n = (int) .999;     // OK???
```

Slide 13

Casting

Duration: 00:01:52
Advance mode: Auto

Notes:

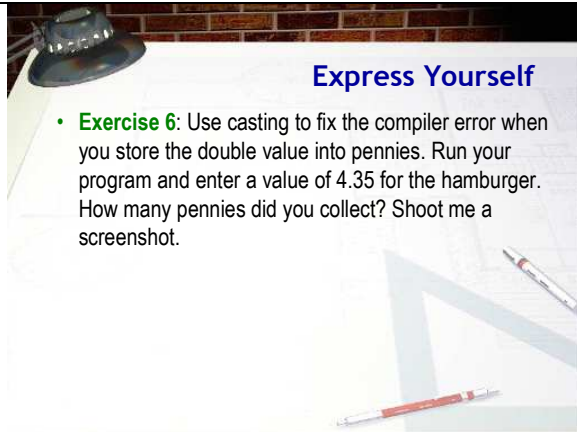
In this last example, when I tried to assign x to the byte variable b, I knew that the value, 123, would fit inside the smaller variable. At other times, even though some data may be lost, the loss may be perfectly acceptable, even necessary.

In those cases, you can tell Java to take its best shot at conversion and store the closest value it can come up with. This is called coercing the type through a "cast" or type cast". When you use a cast, you don't actually change the original value or variable. Instead, a cast creates a new, temporary value that is used in place of the original variable or value.

The syntax for creating a cast is to simply put the type of the new value you want in parentheses, immediately before the variable or value you are trying to cast. Here's my example from the previous slide that stores the value from x into the byte b.

Casting can sometimes be a little confusing. The term, "closest possible value" won't necessarily mean what you think it means. For instance, if you cast the int value 256 to a byte, the compiler won't give you the closest byte value (which would be 127), but will simply transfer the bits into the byte variable, allowing the extra ones to "fall on the floor."

Here's another one. Most of us would think that the closest int to .999 would be 1, but it's not. When you cast a floating-point variable to an int, all of the fractional part is simply discarded. Instead of 1 in this case, you get 0.



Express Yourself

- **Exercise 6:** Use casting to fix the compiler error when you store the double value into pennies. Run your program and enter a value of 4.35 for the hamburger. How many pennies did you collect? Shoot me a screenshot.

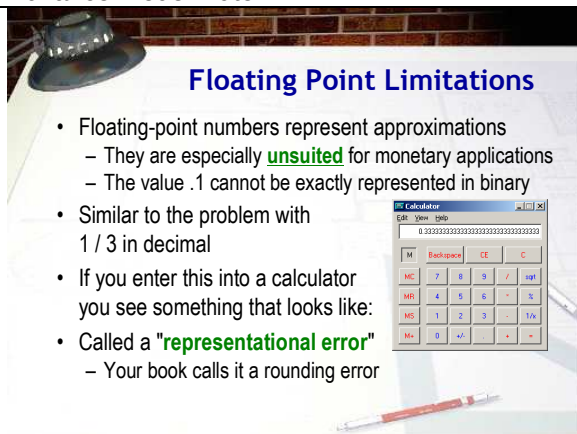
Slide 14 
Express Yourself
 Duration: 00:00:29
 Advance mode: Auto

Notes:

OK, now that you know about casting, you should be able to fix your compiler error. Go ahead and do that. Use a typecast when you store the double value into the variable pennies.

Now, run your program and enter a value of \$4.35 for the hamburger.

Shoot a screen-shot showing me how many pennies you printed out.



Floating Point Limitations

- Floating-point numbers represent approximations
 - They are especially **unsuited** for monetary applications
 - The value .1 cannot be exactly represented in binary
- Similar to the problem with $1/3$ in decimal
- If you enter this into a calculator you see something that looks like:
- Called a "**representational error**"
 - Your book calls it a rounding error

Slide 15 
Floating Point Limitations
 Duration: 00:01:60
 Advance mode: Auto

Notes:

Because of the way that floating point numbers are stored and interpreted, real numbers are often only an approximation of the actual number you are trying to represent. That's especially true when it comes to representing money. Most of the world's currency today is decimal based. Both dollars and Euros have 100 cents. Unfortunately, many common monetary values, like .10, which represents a dime, can't be accurately represented as a binary number.

Although this might seem strange at first, most of us are actually familiar with this concept from fifth and sixth grade when we learned about fractions and decimal numbers. In fifth grade we learned that 1 divided by 3 was $1/3$, and $1/3 + 1/3 + 1/3$ was 1. When decimal numbers came round the next year, however, things got a little more complex; 1 divided by 3 became .3333333 and there didn't seem to be anyway to get back to where you started by adding the results together.

The same kind of thing occurs with binary floating point numbers. Just as you cannot accurately represent the number $1/3$ as a decimal (base 10) number, there are certain numbers that cannot be represented in binary (base 2). The only difference is that there are more repeating fractions with binary numbers than there are in decimal.

This type of problem is called a representational error, or, sometimes, a rounding error.

Floating-point numbers are also susceptible to other kinds of

errors that can snare the unwary or the extra-weary. If you add a very small number and a very large number together, for instance, you will often exceed the number of digits that the floating-point number can accurately represent. You can find out more in the Wikipedia article I linked to at the beginning of this lecture.

A presentation slide titled "What About Money?". It features a background image of a pool table with a cue ball. The slide contains a bulleted list of points and a code block. The points discuss floating-point precision, rounding, and the use of the BigDecimal class. The code block shows how to create BigDecimal objects and perform multiplication.

What About Money?

- What about money, if floating-point isn't that good?
 - We can get closer, by manually rounding the result
 - **Exercise 9:** Try it, re-run and shoot a pic.
- Better, we could create an accurate **Money** class
 - Use **longs** to represent mills accurately
- Java already has a similar class called **BigDecimal**

```
BigDecimal d = new BigDecimal("4.35");
BigDecimal e = new BigDecimal("100");
BigDecimal f = d.multiply(e);
System.out.println(f);
```

Slide 16

What About Money?

Duration: 00:02:34

Advance mode: Auto

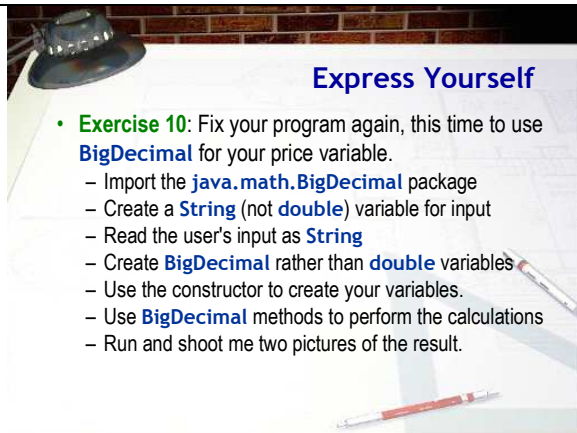
Notes:

So, then, what should we do about money? There are several solutions. First, we could manually "round" the value by adding a half-cent or .005 before converting to an int. To do this, you'll need to multiply by 100 first, then add .005, then wrap the whole expression in parentheses. But the int type-case in front of the expression and then assign that whole thing to the pennies variable.

For Exercise 9, go ahead and make that change and re-run the program. Then, shoot me a picture.

A better solution would be to create an accurate type to represent Money. That's what high-performance financial applications, like those used by stock traders do. Rather than using floating point numbers at all, you can store the actual data and do calculations in thousands or a cent (called mills) and only convert to dollars and cents when doing input and output. The only problem is, since a long can only store numbers up to about ± 18 quintillion, the largest dollar amount you'll be able to represent is about 180 trillion. That's probably enough for your personal financial situation (as long as the dollar doesn't accelerate its devaluation), but it's not enough for things like the national debt. For that, you want an arbitrary-range number system: that is, a number system that mimics the theoretical number system, since it's numbers aren't restricted to a particular amount of storage.

Java has several of these. The one that's best for money is called BigDecimal. To create a BigDecimal object, instead of using BigDecimal literals, you use a constructor that accepts a String representation of the number as a parameter. Here are two BigDecimal objects for the values \$4.35 and 100. Instead of using operators to manipulate BigDecimal objects, you use methods. Here's the multiply method being used to multiply the objects d and e. The product is returned as a new BigDecimal object which here is stored in the variable f. Finally, you can use System.out.println() to print BigDecimal objects, just like you can doubles.



Express Yourself

- **Exercise 10:** Fix your program again, this time to use **BigDecimal** for your price variable.
 - Import the **java.math.BigDecimal** package
 - Create a **String** (not **double**) variable for input
 - Read the user's input as **String**
 - Create **BigDecimal** rather than **double** variables
 - Use the constructor to create your variables.
 - Use **BigDecimal** methods to perform the calculations
 - Run and shoot me two pictures of the result.

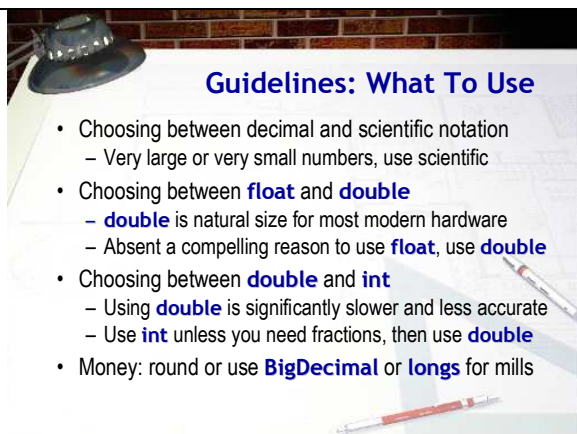
Slide 17 
Express Yourself
 Duration: 00:01:23
 Advance mode: Auto

Notes:

Let's try out using the **BigDecimal** class. Here's all the things you'll have to do.

- To use the **BigDecimal** class you'll need to import it. It's in the **java.math** package.
- You'll need a separate variable for input because there are no routines to read a **BigDecimal** number directly from the user. That means you'll need a **String** variable to hold the user's input before you convert it to a **BigDecimal** value.
- Use the regular **String** input routines to prompt the user and read the input
- Once you have the user's input, you'll need to create **BigDecimal** objects for each number that is used in your program. This includes numbers that were literals in the original program, like 100.
- For each variable, use a **BigDecimal** constructor to create it. Then, use a **BigDecimal** method to perform your calculation. You won't need to do any rounding because **BigDecimal** is perfectly accurate.

When you're done, run the program and shoot me two screen-shots: one of your code and one of the program output.



Guidelines: What To Use

- Choosing between decimal and scientific notation
 - Very large or very small numbers, use scientific
- Choosing between **float** and **double**
 - **double** is natural size for most modern hardware
 - Absent a compelling reason to use **float**, use **double**
- Choosing between **double** and **int**
 - Using **double** is significantly slower and less accurate
 - Use **int** unless you need fractions, then use **double**
- Money: round or use **BigDecimal** or **longs** for mills

Slide 18 
Guidelines: What To Use
 Duration: 00:02:13
 Advance mode: Auto

Notes:

To wrap up this lecture on real numbers, let's go over some guidelines.

First, when you're writing literal values in your code, which should you use, decimal or scientific notation? That depends on the size of the number. Generally, numbers over a million or less than one-millionth should be written using exponential notation, with those in between written using regular decimal notation.

Second, which real-number type should you use most often, float or double? Even though double is larger than float, most modern hardware has special circuitry specifically designed to process double-precision values. Using the smaller-sized float can actually slow down your calculations. The only time you really want to use float values is when there a ginormous number of them, such as a large matrix used in video, imaging or sound, and then the memory savings would be significant. Absent such a compelling reason, use double, not float.

Third, how do you decide whether to use double or int? Many students think things would be easier if there were only one number type, and, since it seems like double can do everything, they want to "standardize" on double. Unfortunately, double is significantly slower and less accurate than int. You really need to use both of them. Ask yourself: "Do I need fractions?" If the answer is no, choose int; if the answer is "yes", then choose double.

	Finally, how about money? If you're writing financial applications you have to be constantly aware of the problems that can arise when using floating-point variables, like double. While you can use floating-point for money, you'll need to round the answers you get back, and knowing exactly when to round and when not to requires quite a bit of mathematical sophistication. A better solution is to use the BigDecimal class, a custom money class, or, in the simplest case, just keep track of pennies or millicents internally, using ints or longs.
--	--